Functional Decomposition and Proposed Program Structure

Identify the major program components and their corresponding responsibilities.

The Rootfinders program will be divided into several major components, each component having its own purpose. These components will work together to calculate and display an approximate real root of a cubic polynomial using the Bisection Method.

User Input Component - Allows users to enter values .

Input Validation Component - Checks if the entered values by the user is valid.

Polynomial Evaluation Component - Calculates the value of the cubic polynomial using the entered coefficients and a given x-value.

Bisection Method Component - This component performs the main numerical calculation needed to find the midpoint, checking the function's value and updating the interval when a stopping condition is reached.

Iteration Recording Component - Records the results of every iteration, including the iteration number, lower bound, upper bound, midpoint, and function value at the midpoint.

Result Display Component - Displays the approximate root, residual, number of iterations, and reason for stopping.

Graphing Component - Displays the graph of the selected cubic polynomial and marks the approximate root position.

Propose the Python functions required for the application.

The functions that will be used for the application are:

get_user_input()

validate_inputs()

evaluate_polynomial(a, b, c, d, x)

check_interval(),

bisection_method() ,

record_iteration()

display_results()

plot_polynomial()

Explain how the identified functions will interact.

How these functions will interact with each other will work in a specific order. The first thing that will happen, get_user_input() will collect the users values. Next, validate_inputs() will check whether the given values are valid. If the user gives an invalid value, it will display an error message.

Once inputs are valid, check_interval() will evaluate the polynomial at the lower and upper bounds. It will check whether the endpoint of the value is already a root or whether the endpoint values have opposite signs. If the interval is valid, the program will proceed to bisection_method(). The bisection_method() will then repeatedly calculate the midpoint and evaluate its function value using the evaluate_polynomial(). It will then update the interval and the function record_iteration() to save the details of each of the calculations.

Lastly, display_results() will then present the calculated root, residual, iteration count, and stopping reason. At the same time, plot_polynomial() will generate the graph of the cubic equation and set/mark the approximate root.

Explain the proposed organization of the application.

The application is split into separate modules so the calculation logic is independent of the GUI. This keeps the code modular, easy to test, and easy to divide among members.

Proposed Program Structure

```
None
numerical-root-finder/
├── main.py # entry point: starts the GUI
├── gui.py # Tkinter window, widgets, event handling
| # get_user_input(), display_results()
```

```
├── validation.py # validate_inputs(), check_interval()
├── polynomial.py # evaluate_polynomial(a, b, c, d, x)
├── bisection.py # bisection_method(), record_iteration()
├── plotting.py # plot_polynomial()
├── presets.py # three built-in equations
├── tests/
│ └── test_bisection.py # acceptance tests + extra edge cases
├── requirements.txt
└── README.md
```

Program flow: main.py launches gui.py. When the user clicks Solve, the GUI collects the inputs and passes them to validation.py. Valid inputs go to bisection.py, which calls evaluate_polynomial() on every iteration and records each step. The GUI then shows the result and the iteration table, and calls plotting.py to draw the graph. Invalid inputs return an error message to the status bar and nothing crashes.

Testing: tests/test_bisection.py covers all five required acceptance tests (known root, noninteger root, endpoint root, bad bracket, a = 0) plus at least two extra edge or invalid-input tests, as required for the testing evidence.